Министерство науки и высшего образования Российской Федерации
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ (НИ ТГУ)
Институт прикладной математики и компьютерных наук

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ 2

по дисциплине «Аппаратная реализация криптоалгоритмов»

Выполнил:
студент группы № 932223
Захаров А.А.

Проверил:
Преподаватель
Разенков С.И.

Томск – 2026

Задание:

1. Определить свой порядковый номер в группе (считать с 1), можно посмотреть в журнале.
2. Если номер делится на 2 без остатка, то придумать автомат Мура, в противном случае - автомат Мили. Нарисуйте его диаграмму.
   Условия:

- разрядность входа автомата равна 8 битам, а разрядность выхода - 1 биту
- выход должен принять значение 1, когда на вход поступило Ваше имя (латинскими буквами в кодировке ASCII)

3. Описать конечный автомат на SystemVerilog.
4. Написать для него тестбенч, выполняющий обход графа переходов, т.е. все имеющиеся переходы из одного состояния в другое. Проверка правильности работы должна выполняться автоматически (assert).
5. Запустить симуляцию и визуально убедиться по временной диаграмме (вейвформе, waveform), что конечный автомат реализован правильно.
   Подсказка: можно увидеть подаваемые буквы, нажав ПКМ на название сигнала на временной диаграмме и выбрав Data Format -> ASCII.
6. Создать проект в Gowin EDA, добавить в него файл с описанием конечного автомата.
7. Выполнить синтез, изучить схематику, посмотреть затраченные ресурсы (Resource Usage Summary) типов Register и LUT, изучить их описание в документе.

```
module snail_fsm (
  input  logic       rst,
  input  logic       clk,
  input  logic [7:0] a,      // ASCII-код текущего символа
  output logic       y
);

  // ASCII-коды букв
  localparam logic [7:0] CH_A = 8'h41,   // 'A'
                         CH_N = 8'h4E,   // 'N'
                         CH_D = 8'h44,   // 'D'
                         CH_R = 8'h52,   // 'R'
                         CH_E = 8'h45,   // 'E'
                         CH_Y = 8'h59;   // 'Y'

  // Состояния — по-прежнему 3 бита (7 состояний)
  localparam logic [2:0] S0 = 3'd0,   // ничего
                         S1 = 3'd1,   // "A"
                         S2 = 3'd2,   // "AN"
                         S3 = 3'd3,   // "AND"
                         S4 = 3'd4,   // "ANDR"
                         S5 = 3'd5,   // "ANDRE"
                         S6 = 3'd6;   // "ANDREY" -> y = 1

  logic [2:0] state, next_state;

  always @(*) begin
    case (state)
      S0: next_state = (a == CH_A) ? S1 : S0;
      S1: next_state = (a == CH_N) ? S2 :
                       (a == CH_A) ? S1 : S0;
      S2: next_state = (a == CH_D) ? S3 :
                       (a == CH_A) ? S1 : S0;
      S3: next_state = (a == CH_R) ? S4 :
                       (a == CH_A) ? S1 : S0;
      S4: next_state = (a == CH_E) ? S5 :
                       (a == CH_A) ? S1 : S0;
      S5: next_state = (a == CH_Y) ? S6 :
                       (a == CH_A) ? S1 : S0;
      S6: next_state = (a == CH_A) ? S1 : S0;
      default: next_state = S0;
    endcase
  end

  always @(posedge clk) begin
    if (rst) state <= S0;
    else     state <= next_state;
  end
  assign y = (state == S6);

endmodule
```

```
`timescale 1ns / 1ns

module tb_snail_fsm;
  parameter TRANSITIONS = 25;

  logic        rst = 0;
  logic        clk = 0;
  logic [8:0]  test_vectors [TRANSITIONS];
  logic [7:0]  a;
  logic        y, y_ref;

  initial begin
    $dumpfile("work/wave.ocd");
    $dumpvars(0, tb_snail_fsm);
  end

  always #5 clk = ~clk;

  snail_fsm dut (
    .rst(rst),
    .clk(clk),
    .a  (a),
    .y  (y)
  );

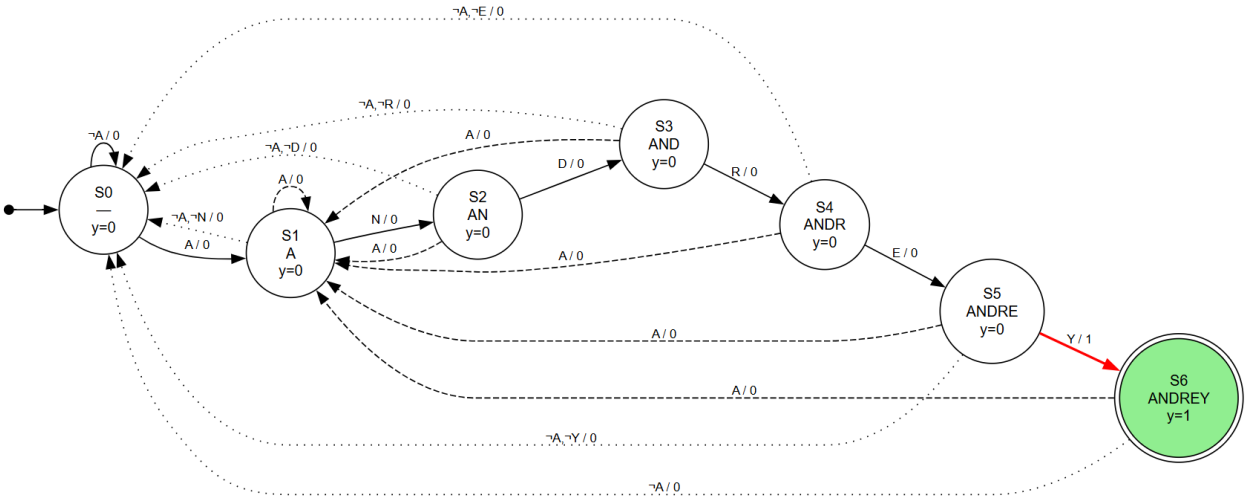
  initial begin
    $readmemb("sequence.tv", test_vectors);

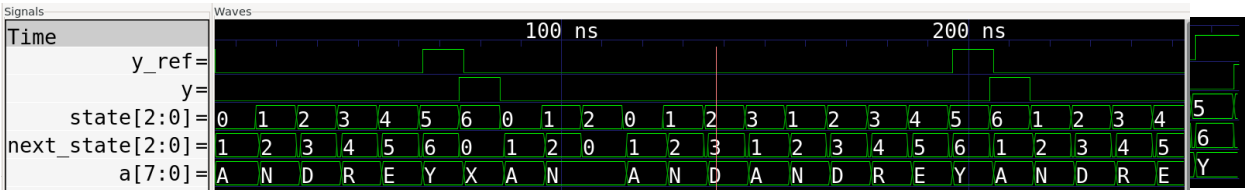
    @(posedge clk); rst = 1'b1;
    @(posedge clk); rst = 1'b0;

    for (int i = 0; i < TRANSITIONS; i++) begin
      {a, y_ref} = test_vectors[i];
      @(posedge clk);
      #1;
      assert (y == y_ref)
        else $error($sformatf(
          "Wrong output on test vector %0d: a = %h, y_ref = %b, y = %b",
          i, a, y_ref, y));
    end

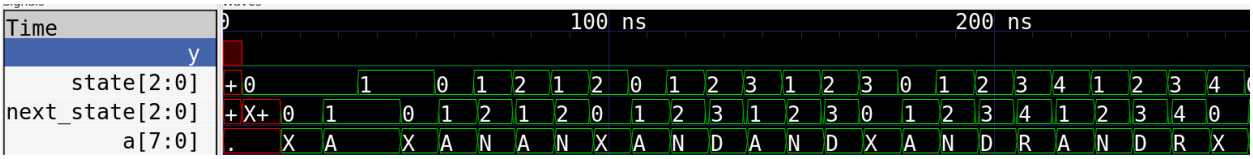
    $display("All %0d tests passed.", TRANSITIONS);
    $finish;
  end

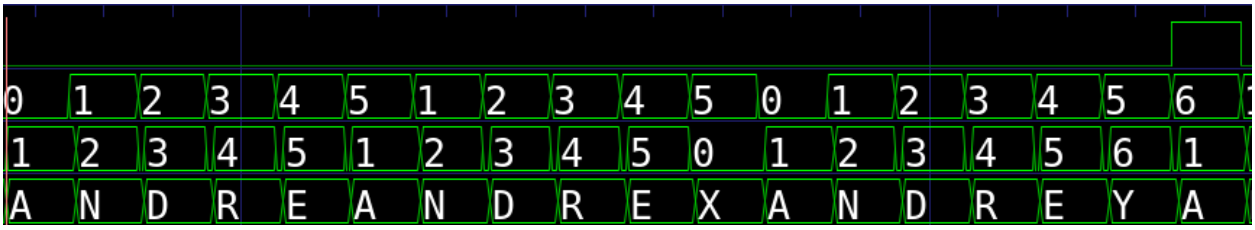
endmodule
```

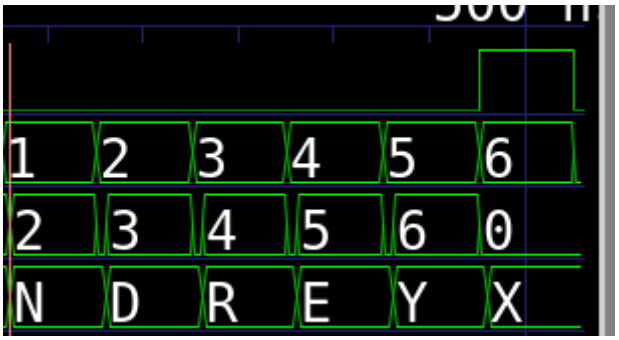

¬A,¬E / 0
¬A,¬R / 0
¬A / 0
¬A,¬D / 0
A / 0
D / 0
R / 0
S0
—
y=0
S3
AND
y=0
S4
ANDR
y=0
¬A,¬N / 0
A / 0
S1
A
y=0
N / 0
S2
AN
y=0
A / 0
A / 0
E / 0
S5
ANDRE
y=0
A / 0
Y / 1
S6
ANDREY
y=1
A / 0
¬A,¬Y / 0
¬A / 0


Signals
Waves
Time
y_ref=
y=
state[2:0]=
next_state[2:0]=
a[7:0]=
100 ns
200 ns
0 1 2 3 4 5 6 0 1 2 0 1 2 3 1 2 3 4 5 6 1 2 3 4
1 2 3 4 5 6 0 1 2 0 1 2 3 1 2 3 4 5 6 1 2 3 4 5
A N D R E Y X A N A N D A N D R E Y A N D R E
5
6
Y

Обход графа


Time
y
state[2:0]
next_state[2:0]
a[7:0]
100 ns
200 ns


0 1 2 3 4 5 1 2 3 4 5 0 1 2 3 4 5 6
1 2 3 4 5 1 2 3 4 5 0 1 2 3 4 5 6 1
A N D R E A N D R E X A N D R E Y A


1 2 3 4 5 6
2 3 4 5 6 0
N D R E Y X

Resource Usage Summary

Resource	Usage
I/O Port	6
I/O Buf	6
IBUF	5
OBUF	1
Register	3
DFFR	3
LUT	9
LUT2	1
LUT3	3
LUT4	5

